

```

• >>> set3.add(5)          #Add a new element
• >>> print(set3)
• {1, 2, 3, 4, 5}
• >>> set3.add(3)          #Add a duplicate element
• >>> print(set3)
• {1, 2, 3, 4, 5}

```

But as a rule of thumb, we should never add mutable objects like list to a set, else we will end up with a `TypeError`.

```

• >>> set3.add([1,2])
• Traceback (most recent call last):
•   File "<stdin>", line 1, in <module>
• TypeError: unhashable type: 'list'

```

- Element deletion from a set can be done in two ways, by using the `discard` function or by using the `remove` function. Both functions take the element to be deleted, as a parameter and delete the same. But if `remove` function is unable to find the element in the `set` it raises a `KeyError` while the `discard` function fails silently.

```

• >>> s = {1, 2, 3}
•
• >>> s.remove(4) #attempt to delete a non-existent element
• Traceback (most recent call last):
•   File "<stdin>", line 1, in <module>
• KeyError: 4
•
• >>> s.discard(4) #attempt to delete a non-existent element
•
• >>> s.remove(3)
• >>> print(s)
• {1, 2}

```

All elements in a given set can be deleted using one function call i.e `clear()` which is called on the `set` that we want to clear.

```

• >>> s.clear()

```